

Research issues in high-throughput distributed object systems

P A Martin

It is an unfortunate fact that current general-purpose middleware implementations cannot support large-scale, wide-area, real-time, high-throughput applications using distributed objects. This paper discusses the background to middleware and explores the need for research into techniques that support high-capacity implementations.

In order to place the role of this work in context, an overview is given of the simple applications-middleware-infrastructure three-layered model that has been adopted. The paper also gives some background on how applications may specify the non-functional characteristics that they demand of supporting middleware, including invocation latency, which is a widely recognised, measured and reported metric, and invocation throughput rate, which is not.

The paper establishes that there is a growing demand for high-invocation-throughput applications, such as the World Wide Web (WWW), collaborative virtual reality (CVR) and intelligent networks (INs), and also that there is increasing availability of suitable high-capacity infrastructure including networks, hosts and operating systems. It also examines the reasons for a lack of implemented high-capacity middleware and reviews current directions in distributed object (DO) research. There is insufficient but growing activity in high-throughput techniques, such as weak-consistency approaches to object replication, that would enable applications to take advantage of high-capacity infrastructure.

1. Introduction

In order to describe the various components that make up a solution, a highly generalised three-layer architecture has been adopted; this is depicted in Fig 1 with example protocols for communication between similar layers on distributed hosts.

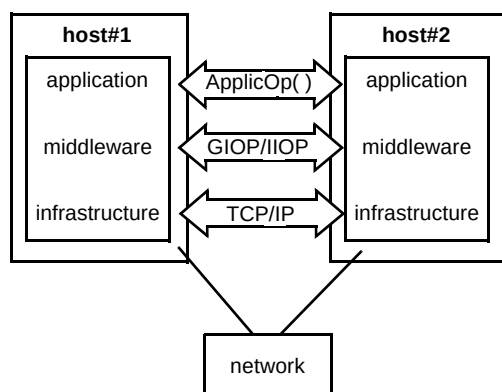


Fig 1 Three-layer architecture overview.

- Application layer

The application layer contains all the logic and data pertaining to the application problem space. Accordingly, it can contain application objects that model real-world entities. For instance, an example IN application may contain ‘customer’ and ‘device’ class objects that model real instances of customers and devices such as ‘Paul Martin’ and ‘his telephone handset’. The communication that passes between application layer entities is application specific and is encapsulated in the protocol of the chosen middleware.

- Middleware layer

The term ‘middleware’ is widely (and vaguely) used in the commercial environment. In the context of this work, it is the layer that provides various ‘transparencies’ to applications. Transparencies enable applications to be written in a manner that is independent of the complexities inherent in distribution [1] and insulated from evolving infrastructures. Transparencies include:

- access transparency, which enables local or remote objects to be accessed in exactly the same manner,
- location transparency, which enables objects to be used without knowledge of their physical location,
- concurrency transparency, which allows several processes to operate concurrently on shared objects,
- replication transparency, which enables objects to be replicated for performance or availability without applications or users having to be aware,
- failure transparency, which enables applications to continue, unaware of the failure of infrastructure elements,
- migration transparency, which enables objects to move within a system without affecting users or applications,
- performance transparency, which refers to the user being unaware of the system configuration in terms of perceived performance,
- scaling transparency, which allows applications to expand without change to application structure or algorithms.

Examples of typical, widely used middleware implementations are the Open Software Foundation's Distributed Computing Environment remote procedure call (OSF DCE RPC) [1] and OmniORB [2] which successfully address access and location transparencies, sometimes referred to together as network transparency. The protocols used at this layer are specific to a middleware implementation and are shown as the Common Object Request Broker Architecture (CORBA) protocols GIOP/IOP (general/Internet inter-orb protocols).

Implementations that fully embody all transparencies will remain a research issue for some years to come. Also, applications vary in their consideration of which transparencies are pivotal. Guidelines for middleware implementors, as to which underlying computer science techniques can be usefully combined in order to satisfy particular application requirements, are desirable but also absent. Accordingly, middleware implementations tend to be general-purpose and hence non-optimised to specific applications' non-functional requirements.

- Infrastructure layer

The infrastructure layer contains components such as networks, routers, hubs, hosts, protocol stacks and operating systems. These are constantly evolving elements, often with their own specialised application programming interface (API), such as inter-process communications (IPC) APIs. Accordingly, all

application code that directly uses these APIs must be ready to evolve as new elements are introduced. It has been necessary for some high-throughput applications to access these low-level, minimum-overhead APIs in order to meet their performance requirements.

2. High-throughput applications background

Firstly, this section describes the background to high-throughput applications by discussing existing approaches to the World Wide Web (WWW), collaborative virtual reality (CVR) and intelligent network (IN) deployment and their attendant shortfalls. A number of approaches have been devised to support the throughput requirements of distributed applications. Examples are the proxy caching employed in WWW servers, tolerance of inconsistent replicas in CVR, and the 'full-replication' approach of IN services. Secondly, the non-functional characteristics that applications can use to specify their demands on supporting middleware and infrastructure are described. Lastly, the issues that prevent applications from taking full advantage of modern high-capacity infrastructure are listed.

2.1 Intelligent network applications

IN applications are triggered during the set-up of telephone calls and large service times result in end-user dissatisfaction. Taking as an example the call-screening IN application, it is possible to see how the scalability and throughput issues have been addressed in the face of stringent timeliness criteria. A typical current implementation would support 10 000 customers. The call-screening service enables subscribers to decide on which end-point to terminate a call attempt, on the basis of who the caller is. For example, a call-screening customer, such as 'Paul Martin', may direct all calls from non-preferred callers to a voice-messaging system and all calls from a preferred caller to their mobile phone. Figure 2 depicts how such a service may be deployed on a number of computing sites.

Assume that the caller, Chris Gittins, has dialled the number of the callee, Paul Martin. The caller now awaits some tone to indicate if the call has been successfully routed or not. The IN-enabled telephony switch will recognise that the dialled number requires special routing and will generate a message, which it will forward to a computing site.

A site is generally a cluster of high-capacity symmetric multi-processor (SMP) computers with service applications that can respond to the incoming message. In this case there is a call-screening application running that receives the message, determines the calling and called parties from fields in the message and invokes the appropriate function to determine the terminating device. A reply message is generated and sent to the switch, which can then route the outstanding call attempt to the chosen terminating device.

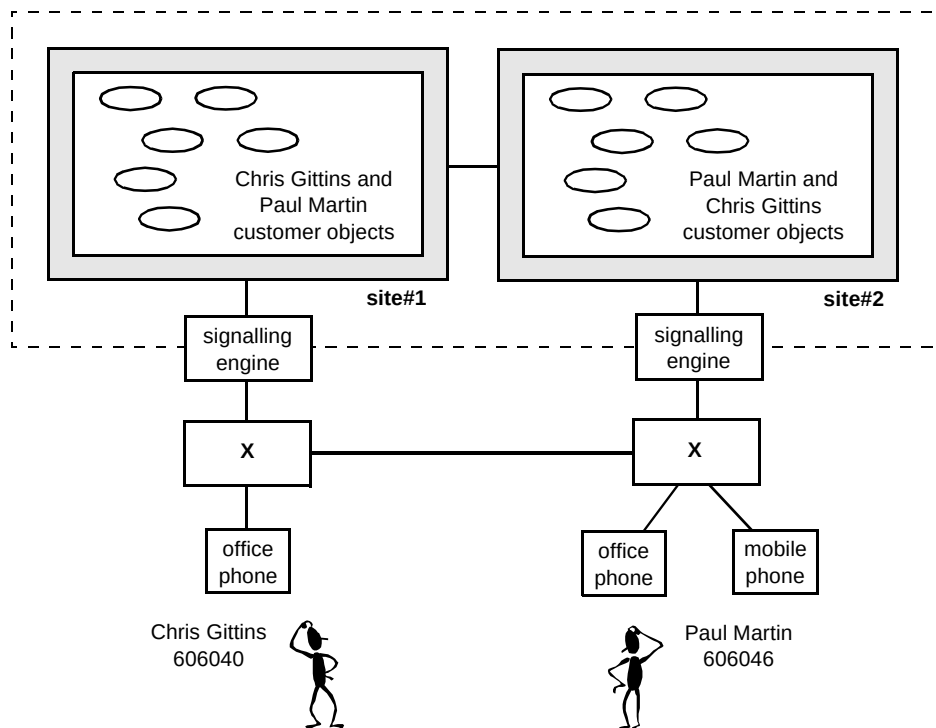


Fig 2 Call-screening IN application scenario.

Eventually, the remote switch (connected to the terminating device) will generate a 'distant ringing', 'device busy' or a 'device unobtainable' tone which is relayed back to the caller.

The SMP hosts are referred to as 'fully replicated' as they have a full copy of both the service logic and all the read-only customer data in RAM on every host. In order to satisfy a site's throughput requirements, there will be a sufficient number of fully replicated hosts in the cluster, each handling a fraction of the site's calls. Updates to the application data are managed separately by a centralised database server that periodically, sometimes only daily, downloads updates to the hosts.

The foregoing was a brief description of a single call attempt for a specific IN service, which should take less than one second to perform. Such services, with current full-replication implementation techniques, scale to only thousands of users, which prevents their global seamless deployment.

Within the context of IN applications the Telecommunications Information Networking Architecture Consortium (TINA-C [3]) group has made the most progress. TINA-C is an industrial consortium that encourages the necessary research and standardisation to enable global interoperable IN and other applications. Significant work has been done with a number of Object Request Brokers (ORBs) that have proven interoperability; however, very little in-depth work has been done to demonstrate throughput and scalability. TINA white papers have identified the need for optimised ORBs to satisfy the non-functional requirements of tele-

communications applications, including massive scalability and throughput, but as yet no implementation approaches have been proven.

2.2 Internet applications

No discussion of wide-area, scalable, high-throughput applications would be complete without some treatment of Internet applications such as e-mail, news and the WWW. The recent explosion in the accessibility of the Internet, and the WWW that runs over it, has led to some interesting approaches to satisfying the throughput demands of such applications. It is a well-known feature of popular sites that request latency can be excessively high due to network and computational overload. Currently some of this overload is alleviated by the use of caches which can be in the browser, server, or between the two, in proxy-servers. Caching proxy-servers provide a read-only cache of recently accessed Web pages and provide improved overall performance, but, in common with any replication approach, introduce the possibility of inconsistency.

Current access patterns to Web resources are strongly read-dominated allowing servers to be stateless, but the advent of new applications such as Web-based electronic commerce and dynamic documents may change this simple model, which impacts upon the approaches to handling consistency in the caches. Leading research projects, such as Globe [4], seek to provide frameworks for supporting WWW applications via distributed, shared, object-oriented (OO) approaches and propose that consistency-policies be applied on a per-object basis.

2.3 Collaborative virtual reality applications

Recent years have shown a growing research interest in scalability issues associated with supporting virtual reality (VR) worlds. Leading projects include DIVE [5] and MASSIVE [6] that seek to support thousands of users with hundreds of thousands of simulation entities. They have exploited a number of methods for controlling the computational and network load of adding additional users and entities. Firstly, they have exploited applications semantics by restricting entity state updates to be delivered only to those parties that have expressed an interest rather than to all parties. Secondly, they have determined that strict consistency between replicas is difficult to combine with real-time interaction and have accepted some inconsistency tolerance in order to maintain rapid response times.

In the case of DIVE, the relevant parts (called sub-hierarchies) of the full-world database are replicated in each interested process so that data can be accessed from memory, keeping latency low. Interestingly, they bypass the traditional distributed database approach of ensuring all processes agree on the information's state before committing updates. Changes are made to the local in-memory copy of the database, then marshalled and distributed to other interested hosts where updates are applied. The inconsistency introduced is strongly dependent on the network latency, reportedly up to 800 ms. Their experience of virtual worlds is that entities are edited in bursts and then stabilised, and differences due to network latency and packet loss are then 'healed' over time by periodic synchronisation with entity version numbers and possible update requests.

3. Specification of future application demands

A number of techniques are available for the specification of an application's non-functional requirements, and the most suitable and popular seems to be real-time UML (Unified Modelling Language) [7]. However, UML currently lacks the notation to specify some non-functional QoS-related characteristics, such as method invocation throughput rates, which must be added as 'notes' to UML diagrams.

The application requirements that need to be specified, along with their metrics, are given in the following sections.

3.1 Throughput demand

It is expected that the global demand for IN 'transactions' will dramatically increase from the current 50 000 per day on one carrier's network, up to a proposed 5000 000 000 per day by the year 2003 [8]. With complex applications, nested invocations become necessary and the total invocation throughput becomes approximately

100 000 per second. Throughput in the WWW is dramatic with some sites experiencing millions of hits per day.

Throughput is a pivotal characteristic as revenue is potentially generated from individual accesses. It is important to note that throughput in this context is different from the widely accepted metric of bytes-per-second for measuring ORB performance when, for instance, passing large arrays of integers in parameters. To avoid confusion, the term invocation throughput will be used. Current middleware reports are not sufficiently wide-ranging to address invocation throughput as a significant metric, despite this being the central indicator of the capacity of a system.

3.2 Invocation latency

The invocation latency, measured by the simple metric of milliseconds (or more recently microseconds), may not exceed the order of 100 ms for IN applications to satisfy the total end-to-end service time of 1 second. This does not seem particularly demanding in isolation but becomes difficult when combined with other wide-area and scale criteria. For CVR, the maximum latency tolerated is 100 ms to maintain the illusion of reality. In the WWW, minimal latency is clearly preferable to reduce the time spent waiting for pages.

3.3 Access patterns

The applications considered are strongly read-dominated, but must support some editing and update activity. For instance, updates to Web pages stating a change in a meeting venue will need to be 'pushed' to proxy-servers and perhaps browser caches as well. In IN applications there is also a demand to customise (or update) service data for individual usage.

A metric that has been widely applied to access patterns is the ratio of read to write operations. It is felt that, as more detail is required, the effect of each read and write operation needs to be specified. This allows more informed and fine-grained replication and caching decisions to be made. Each operation could define the number of data bytes read and/or edited per invocation and specify the percentage of total invocations that are directed to that operation.

It is also notable that the access patterns of these three applications demonstrate some popularity or 'temporal locality' effect which can be employed in partial replication or caching decisions. For instance, CVR implementations employ sub-hierarchies and have observed that entities are accessed in bursts. Similarly, Web caches primarily store the most popular pages. In the case of IN, it has been observed that the vast majority of an individual customer's call attempts are to a limited 'hot-list' of numbers.

3.4 *Inconsistency tolerance*

A common characteristic of the target applications is that they are prepared to trade off data consistency for rapid response times, e.g. WWW cached pages can potentially be a day out of date. The specification of inconsistency tolerance is a research issue as it does not have a widely understood metric, and, additionally, a confusing number of consistency policies are to be found in the literature. For the moment, it should be assumed that a simple tolerated 'staleness' time can be attached to data items.

3.5 *Global access*

IN applications are accessible from any of the billions of world-wide PSTN end-points. WWW access is also increasingly ubiquitous, but still numbers its access points in millions. CVR applications have more restricted access points but are addressing global distribution. Global access is here assumed as a given criterion, which makes the minimisation of WAN latency an important topic.

3.6 *Large numbers of application objects*

The Web is expected to eventually support billions of globally distributed application objects. Also, CVR applications, such as military simulation, are expected to support hundreds of thousands of entities (objects) by the year 2000. No evidence could be gathered on the size of these objects in terms of memory demand, but it is safe to assume they will only grow.

To further support the growth in object count scalability and throughput requirements, consider a 'personal numbering' IN application, which requires that numbering is associated with individual customers rather than end-point devices. The UK Telephony Regulator, Oftel, in order to enable free trade by allowing customers to move between operators, requires that personal numbering becomes increasingly available. If this application could be enabled, all calls will potentially become IN calls, leading to a demand for a throughput of 3000 IN 'transactions' per second involving tens of millions of application customer objects in the UK alone.

The specifications of other criteria, such as availability, are beyond the scope of this paper.

4. **Application issues**

The foregoing was a description of three applications and their anticipated non-functional demands. They have some unifying characteristics which include being user-event driven (and hence with interactive latency requirements), wide area access, high-throughput demand, some data inconsistency tolerance, strongly read-dominated access patterns, and large scalability requirements in terms

of numbers of users and objects. There are working implementations of the target applications but they are restricted by the following issues.

4.1 *Lack of suitable middleware*

It is recognised that there is currently a lack of suitable middleware to support this family of applications [9]. This has led to programmers having to repeatedly devise implementation solutions. For example, in a non-distributed implementation, it is likely that application code would call run-time library and operation system (O/S) functions directly to provide all the services, such as time and file services, it requires.

In the applications of interest to this work, it is the general but unfortunate trend to enable distribution by continuing to use O/S primitives directly [9, 10], such as inter-process communications (IPC). This is done in order to attain a higher throughput but has a side-effect of losing the benefits of an insulating reusable middleware layer.

There is a research issue to address this undesirable trend and embody the underlying techniques employed in successful application implementations into suitable middleware that is reusable across domains.

4.2 *Existing approaches require evolution*

The implementation approaches described above satisfy their current application's needs, but they all require evolution to meet future needs.

The CVR approaches of entity sub-hierarchies seem promising, but are targeted on object counts up to hundreds of thousands rather than millions or billions of objects. Current approaches to Web proxy caching are workable but are read-only document-based rather than editable computational object-based. New approaches based on dynamic data are proposed but as yet unproven.

To focus on an IN example, the traditional full-replication approach has proved to be necessary to satisfy the stringent throughput and real-time requirements of IN applications, but it has inherent scalability limitations. Firstly, the customer data is fully-duplicated on every host, which means that the whole population which an IN service can support is limited to how many customer data records can be fitted in a single host (typically only of the order of 100 000). Furthermore, as new customers are added, the RAM to store them has to be purchased for each and every host, typically tens of machines. These effects prevent using full-replication to enable scalability to ubiquitous, global IN services. Moreover, the duplication effect is compounded as an IN service is typically scaled by fully replicating a site. Inevitably, while enabling an economically tolerable throughput, this approach leads to a limitation on the

number of customers that can be supported by IN services. Secondly, full-replication leads to a restrictive overhead for data management as each update will lead to a promulgation to each site and from there to each host. This leads to a large and expensive WAN bandwidth demand.

In summary then, it is clear that the increasingly high demands for complex, customisable, high-throughput, large-scale, global-access applications will be prohibitively expensive to satisfy using existing approaches. Accordingly, distributed OO approaches become more attractive. However, existing implementations of OO middleware cannot satisfy the scalability and throughput requirements of anticipated applications.

5. Infrastructure background

Infrastructure, with reference to Fig 1, consists of the O/S, host computers, programming language and run-time support, protocol stacks and network apparatus. Recent years have seen the rapidly increasing capacity of computing systems for rapidly decreasing expenditure. Particularly, there has been a rapid decline in the cost of RAM, enabling memory-hungry approaches to implementation to become more acceptable. Also there has been an increasing availability of Gigabyte per second LANs, and also high bandwidth WANs. These changes have forced BT to review previous assumptions about how to construct high-throughput distributed systems.

Current implementations of high-throughput distributed systems have been limited by low-bandwidth or prohibitively expensive WANs leading to an assumption that moving application state between sites is a costly and slow operation. Also, existing hosts are often based on traditional Unix platforms where many processes are executed on a uni-processor host. This incurs a high process-context switch overhead leading to constrained throughput. Given the nature of the current infrastructure, these middleware approaches are now clearly far from optimal. A number of important trends have been isolated.

- Programming language developments

Increasingly distribution is becoming part of evolving languages such as Java and Ada9x. In the case of Java, concurrency control is part of the language and also the Java code component can be transmitted across a network and executed at a remote host. In addition, the serialisation of Java objects allows data to be transmitted, although the efficiency of current implementations requires some optimisation.

- Low-cost, high-throughput computers

Commodity SMP clusters represent a cost-effective platform for scalable, parallel computation. Compared to uni-processors, SMP nodes provide the economic

advantage of packaging multiple processors together in a single chassis, and the technical advantage of high-speed, tightly coupled communication between processors within the SMP.

- High-bandwidth LANs and increasing bandwidth WANs

As previously stated Gbit/s LANs and Mbit/s WANs are now commonplace. However, as shown in the next section, middleware implementations are often evolutions of designs that are optimised for 10 Mbit/s Ethernet constraints.

- Large address spaces and real-time O/S

The advent of 64-bit processors and operating systems have relieved the programmer of the potential address-space exhaustion issue and opens the door to single address-space computer research directions [11]. Also, mainstream operating systems are increasingly introducing real-time features, such as access to scheduling and timed condition variables.

- Multithreading

As hardware costs continue to decline there is a clear trend towards lightweight processes and parallel processing and there is also some evidence to suggest that multithreading is necessary to realise continuing performance gains [12]. Additionally, applications that must handle asynchronous events are easier to design using multithreading. Multithreading coding is not currently widespread as it is complex and has a limited pool of skilled programmers. Accordingly, to address the complexity issue, implementors of Java have considered multithreading sufficiently important to build synchronisation syntax into the language. It is expected that the trend towards multithreading will become more prevalent.

These trends show that high-capacity hardware and software infrastructure is becoming increasingly available at economical rates.

6. Middleware background

The term 'middleware' has only recently come into general recognition for the software component that includes enabling distribution. Before this, the components existed, but without the unifying 'middleware' title, and, accordingly, projects that have been focused on topics such as distributed objects and remote procedure calls (RPCs) are considered relevant background. Very extensive research work has been carried out in recent years to exploit the clear synergy between object-oriented approaches and distribution, and a major goal has been to achieve the distribution transparencies described above.

Initial projects allowed programmer access to IPC and message-passing primitives requiring application programmers to undertake marshalling directly, and distribution was therefore explicit. An evolution, exemplified by OSF DCE [1] introduced a level of distribution transparency by using definitions of interfaces via specialised languages called interface description languages (IDLs). This enables the marshalling components of distributed applications to be automatically generated. A further evolution of this approach has been the adoption of 'proxies' which are representations of remote objects in local address-space. Proxies are widely used in implementations of CORBA and allow invocations to be programmed in the same manner for local invocation and remote invocation, thereby becoming network transparent.

These systems all use forms of RPC or RMI that require the underlying system to perform message passing to execute operations on remote objects, thereby always incurring network latency. These mechanisms have been the subject of significant study and are now well understood.

6.1 Background to large-scale high-throughput middleware

The foregoing has briefly described the generalised use of middleware, which has proved to be acceptable for the vast majority of applications. However, the main interest is in supporting the currently under-addressed applications that have stringent timeliness, scalability and throughput requirements.

Real-time embedded systems have been used in such applications for about 50 years. Such systems have often evolved from specialised hardware and software components and are supported by their own distinct research. Another requirement for the target applications is to access large amounts of data, typically using databases. Database technology is another distinct area of computer science research. The research diversification between real-time and database topics has led to a diversification of commercial off-the-shelf (COTS) products and accordingly there are no unified products that support stringent timeliness criteria as well as accessing large numbers of data items. Given this existing gap it is unreasonable to expect that the introduction of a third new research discipline, that of distributed objects, can rapidly fill it.

Therefore the tendency has been for applications with stringent latency and throughput requirements, to adopt *ad hoc* middleware approaches to produce deployable applications. *Ad hoc* approaches to middleware, however necessary, have a number of drawbacks including the typical requirement that programmers have intimate knowledge of low-level IPC primitives, such as TCP/IP sockets. This unfortunate trend is now recognised and is being addressed by the middleware standards community.

6.2 CORBA-related activity

Standardisation efforts have led to standards such as CORBA [13] being widely recognised. A major goal of middleware, and CORBA in particular, is 'interoperability' which has been fully addressed for multiple host architectures, programming languages and operating systems. The Object Management Group (OMG), that generates CORBA standards, has over 800 member organisations from widely varying application domains, such as banking, medical, telecommunications and avionics. Accordingly, the standards proposed by such a wide member base are necessarily generalised. Also, the much smaller group of CORBA implementors must also agree standards as they evolve, which is again an influence for generalisation.

CORBA is widely accepted as an influential distributed object (DO) architecture and has generated many implementations. The architecture was initially defined by five pioneering and competing implementors and has now evolved into full standards that have proven interoperability between implementations. Notable achievements include the definition of a programming-language-independent IDL and the definition of the IIOP.

The Object Management Architecture (OMA) activity intersects our interests at a number of points. In addition to core ORB work, there are technical working groups focused on events, real-time CORBA and minimum CORBA, which defines the tolerable subset of the full CORBA functionality, thereby enabling the implementation of reduced footprint ORBs, primarily for embedded applications. Also, there are task forces working in the telecommunications arena, mostly defining flow control or streams interfaces, and the IN arena, mostly developing IDL representing the existing telephony SS7 (signalling system no. 7) and INAP (intelligent network application part) protocols. Currently there is no work that enables the interworking of service control points from different enterprises.

In CORBA systems, the specification of QoS for object interaction, as opposed to the control of multimedia streams, is a recently addressed research issue and as yet no feedback is available on implementation results.

7. Middleware trends and issues

7.1 Customisation — 'one size does not fit all'

It is widely recognised in the CORBA community that 'one size does not fit all' which means that one implementation of the CORBA standard will not match the non-functional characteristics of all applications. For example, a generalised CORBA implementation that is adequate for a low-throughput, high-security, interoperable banking application, is unlikely to also be suitable for a

very high throughput, large-scale, constrained-latency IN application. This trend is leading to the 'componentisation' of middleware implementations where application-optimised code can be integrated with the core ORB. Examples of components include 'smart proxies', object location mechanisms, and object and method demultiplexing mechanisms. Another leading research implementation, DIMMA [14], is highly customisable where components even include the threading model. However, there is little design guidance available for programmers in the appropriateness of which mechanisms best match which combination of application requirements.

7.2 Replication

A second and vital research trend has been the study of replication approaches. In order to address the latency and availability drawbacks of single-object-copy approaches, many researchers have proposed replicating objects on multiple hosts [4]. Replication is widely recognised as the key to providing good performance and high-availability in distributed systems. Using replication, method invocations may be satisfied locally if a replica is present or will be forwarded to the 'closest' replica.

The main problem with replicated object systems is that of applying operations from clients to multiple replicas in a consistent way, while maintaining reasonable response times and system throughput rates. A range of consistency models have been explored from the weakest being an 'asynchronous' to the strongest being a strict 'totally synchronous' model.

In the asynchronous model, all requests are served by the local server and updates are promulgated when convenient or periodically, and updates are processed by servers as they arrive. This model has the advantage of maximising response times because, if the replicated data is read-only or if consistency is not an issue, response times are potentially the same as non-replicated local responses, although the sending and handling of updates in the background would have some impact.

Alternatively, in the totally synchronous model all updates are 'totally ordered' so that strict consistency is maintained. The attendant disadvantage is that throughput is reduced due to the overhead of ordering. A number of schemes have been devised that achieve satisfactory trade-offs between these two extremes including quorum-based schemes and causality.

As the target applications can tolerate a degree of inconsistency, strict consistency models are not necessary. Also, the pivotal issue for the applications is throughput, which strongly directs current interest towards weak-consistency models.

8. Conclusions

It can be seen that the demand for applications, such as WWW, CVR and IN services that have a global distribution of millions or billions of objects with a high-throughput of low-latency invocations, will grow significantly into the next millennium. It is also clear that the network and computing infrastructure capabilities are growing rapidly to support high-throughput and low-latency communications and computing. However, many existing middleware implementations do not take full advantage of modern memory- and computation-rich hosts, and high bandwidth network infrastructures. Moreover, it is felt that high-performance and scalability middleware research issues and optimisations for the target applications and infrastructure are insufficiently addressed.

Significant research and standardisation effort has already been expended to address distributed object performance. RPC-style approaches are widely implemented and their performance and scalability drawbacks are well understood. The key to wide-area, high-performance distributed-object support is recognised to be replication and caching, but few such systems exist as yet and design trade-offs are not well understood. The central debate with replication approaches is control of replica consistency without prohibitive performance loss. Current interest in high-throughput solution points to weak-consistency approaches. In order to guide middleware implementors, firstly, application characteristics must be specified and secondly, requirements must be matched to appropriate implementation techniques. Application characteristics include throughput, access patterns, latency, object count and size, geography, concurrency, data consistency and resource constraints.

Unavailability of suitable middleware implementations, allied to the necessity of satisfying throughput and timeliness criteria, has led to a continuing deployment of non-scalable techniques and use of low-level IPCs and O/S functionality. Accordingly, programmers are forced to implement non-uniform object models and lose the benefits of an insulating middleware layer.

It is clear then that research is required to understand how to optimise replicated-object approaches in order to support forthcoming application demands in the face of increasing infrastructure capacity.

References

- 1 Lockhart H W: 'OSF DCE guide to developing distributed applications', McGraw-Hill (1994).
- 2 Lo S-L and Pope S: 'Implementation of a high-performance ORB over multiple transport networks', Proc of Middleware '98, Springer (1998).

- 3 Dupuy F, Nilsson G and Inoue Y: 'The TINA consortium: toward networking telecommunications information services', IEEE Communications Magazine (November 1995).
- 4 van Steen M R, Homburg P C and Tanenbaum A S: 'The architecture of Globe: a wide-area distributed system', Vrije Universiteit, Internal Report IR-0422 (March 1997).
- 5 Frecon E and Stenius M: 'DIVE: a scalable network architecture for distributed virtual environments', Distributed Systems Engineering, 5, No 3 (September 1998).
- 6 Greenhalgh C: 'Awareness-based communication management in the MASSIVE systems', Distributed Systems Engineering, 5, No 3 (September 1998).
- 7 Douglass B P: 'Real-time UML: Developing Efficient Objects for Embedded Systems', Addison-Wesley (1997).
- 8 Kim H H: 'Intelligent network services in the next decade', IN Symposium Estoril '98 (1998).
- 9 Schmidt D C et al: 'A high-performance end system architecture for real-time CORBA', IEEE Communications Magazine (February 1997).
- 10 Bogunovic N: 'Visual resource management in heterogeneous distributed computing environments', Proc of the 10th IASTED PDCS Conference (1998).
- 11 Mullender S J: 'Kernel support for distributed systems', in Mullender S J (Ed): 'Distributed Systems: Concepts and Design', 2nd Edition, Addison-Wesley (1993).
- 12 Northrup C J: 'Programming with Unix threads', John Wiley & Sons (1996).
- 13 Vinoski S: 'New features for CORBA 3.0', Communications of the ACM, 41, No 10 (October 1998).
- 14 Donaldson D et al: 'DIMMA — a multi-media ORB', Proc of Middleware '98, APM Ltd (1998).



Paul Martin is the Technology Champion for the Network Intelligence Engineering Centre within BT Laboratories at Martlesham Heath, Suffolk in the UK.

He has previously worked on network computing, large-scale IN, CTI and 'voice over IP' projects.

Before joining BT, he worked in the aeronautical and software industries. He is a Chartered Mechanical Engineer and a PhD candidate at Essex University.